

API Documentation

1. Project Information

Item	Description
Project Name	NOT-TEMU
API Type	REST API
Backend Framework	Spring Boot
Database	PostgreSQL (Supabase)
ORM	Hibernate / Spring Data JPA
Deployment	Render
Base URL (Production)	https://not-temu.onrender.com
Base URL (Local)	http://localhost:8081

2. Authentication API

Register User

Item	Value
Method	POST
Endpoint	/auth/register
Description	Registers a new user

Request Body

```
{  
  "name": "Khoa",  
  "email": "khoa@test.com",  
  "password": "123456"  
}
```

Success Response

```
{  
  "message": "User registered successfully"  
}
```

Error Responses

Status Code	Meaning
400	Missing required fields
409	Email already exists
500	Internal server error

Login User

Item	Value
Method	POST
Endpoint	/auth/login
Description	Authenticates user login

Request Body

```
{  
  "email": "khoa@test.com",  
  "password": "123456"  
}
```

Success Response

```
{  
  "message": "Login successful"  
}
```

Error Responses

Status Code	Meaning
400	Invalid request
401	Incorrect credentials
500	Internal server error

3. Product API

Get All Products

Item	Value
Method	GET
Endpoint	/products
Description	Returns all products

Example Response

```
[  
  {  
    "id": 1,  
    "name": "Gaming Laptop",  
    "description": "RTX Laptop",  
    "price": 1200,  
    "stockQuantity": 5  
  }  
]
```

Get Product By ID

Item	Value
Method	GET
Endpoint	/products/{id}
Description	Returns a single product

Error Responses

Status Code	Meaning
404	Product not found

Create Product

Item	Value
Method	POST
Endpoint	/products
Description	Creates new product

Request Body

```
{  
  "name": "Mechanical Keyboard",  
  "description": "RGB keyboard",  
  "price": 99.99,  
  "stockQuantity": 10  
}
```

Error Responses

Status Code	Meaning
400	Invalid input
500	Internal server error

Update Product

Item	Value
Method	PUT
Endpoint	/products/{id}
Description	Updates product details

Delete Product

Item	Value
Method	DELETE
Endpoint	/products/{id}
Description	Deletes a product

4. Order API

Checkout / Create Order

Item	Value
Method	POST
Endpoint	/orders/checkout
Description	Creates a new order

Request Body

```
{
  "userId": 1,
  "items": [
    {
      "productId": 1,
      "quantity": 2
    }
  ]
}
```

Success Response

```
{
  "orderId": 10,
  "status": "CREATED"
}
```

Validation Rules

Validation	Description
Product exists	Invalid IDs rejected
Quantity > 0	Prevents invalid orders
Sufficient stock	Prevents overselling

5. Payment API

Process Payment

Item	Value
Method	POST
Endpoint	/payments/pay
Description	Simulates payment processing

Request Body

```
{  
  "orderId": 10,  
  "method": "CARD",  
  "cardNumber": "4111111111111111",  
  "cardBrand": "VISA"  
}
```

Success Response

```
{  
  "status": "SUCCESS",  
  "transactionId": "TXN-938293"  
}
```

Notes

Feature	Status
Real payment gateway	Not implemented
Card validation	Basic simulation
Transaction ID	Auto-generated
Payment persistence	Implemented

6. Standard Error Format

Example Error Response

```
{  
  "timestamp": "2026-05-13T12:00:00",  
  "status": 404,  
  "error": "Not Found",  
  "message": "Product not found"  
}
```

7. Validation & Error Handling

Feature	Implemented
Duplicate email prevention	Yes
Invalid login handling	Yes
Empty field validation	Yes
Product existence validation	Yes
Stock validation	Yes
Global exception handling	Yes

8. Database Models

User

Field	Type
id	Long
name	String
email	String
passwordHash	String
role	String

Product

Field	Type
id	Long
name	String
description	String
imageUrl	String
price	Double
stockQuantity	Integer

Order

Field	Type
id	Long
orderAmount	Double
orderDate	Timestamp
status	String
userId	Long

OrderItem

Field	Type
id	Long
quantity	Integer
purchasedPrice	Double
orderId	Long
productId	Long

Payment

Field	Type
id	Long
method	String
status	String
transactionId	String
cardBrand	String
cardLast4	String
walletProvider	String
orderId	Long